

PE-EC703A

Embedded System

Combined Module Notes

Modules 1 – 5

Semester VII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: What is an Embedded System?	5
1 What is an Embedded System?	5
1.1 Key Characteristics	5
2 Embedded Processor in a System	5
2.1 Types of Embedded Processors	6
3 Components of an Embedded System	6
3.1 Hardware Components	6
3.2 Software Components	7
4 Embedded Software in a System	7
4.1 Levels of Embedded Software	7
4.2 Brief Introduction to Embedded Software Categories	8
5 Design Process in Embedded Systems	8
5.1 Design Steps Explained	8
5.2 Hardware–Software Co-design	9
Module 2: 8051 Microcontroller — Architecture Overview	13
1 8051 Microcontroller — Architecture Overview	13
1.1 Key Features of 8051	13
1.2 Real-World Interfacing of 8051	14
2 ARM Processor Architecture	14
2.1 ARM vs. 8051 — Key Differences	15
2.2 ARM Processor Features	15
3 Processor and Memory Organization	15
3.1 Memory Types in Embedded Systems	16
3.2 Harvard vs. Von Neumann Architecture	16
3.3 Memory Map of a Typical MCU	16
4 Parallelism in Instruction Level	17
4.1 Pipelining	17
4.2 Pipeline Hazards	18
5 Processor and Memory Selection	18
5.1 Memory Selection Criteria	18
Module 3: Serial and Parallel Communication Ports	22
1 Serial and Parallel Communication Ports	22
1.1 UART (Universal Asynchronous Receiver/Transmitter)	22
2 Timer and Counting Devices	23
2.1 8051 Timer Modes	23

3	Watchdog Timer	23
3.1	Watchdog Timer Operation	24
4	Real-Time Clock (RTC)	24
5	Serial Bus Communication Protocols	24
5.1	I2C (Inter-Integrated Circuit) Protocol	25
5.2	CAN (Controller Area Network) Protocol	25
5.3	ISA (Industry Standard Architecture) — Parallel Bus	26
6	Interrupt Service Mechanism	26
6.1	ISR (Interrupt Service Routine)	26
6.2	8051 Interrupt Sources	27
6.3	Different Interrupt Sources (General)	27
7	Multiple Interrupts, Interrupt Latency and Deadline	27
7.1	Multiple Interrupt Handling	28
7.2	Interrupt Latency	28
7.3	Interrupt Deadline	28
7.3.1	Classification of Interrupt Deadlines	29
7.3.2	Key Factors Causing Deadline Violations	29
7.3.3	Design Strategies to Meet Deadlines	29
Module 4: Assembly Language Programming (ALP) in Embedded Systems		33
1	Assembly Language Programming (ALP) in Embedded Systems	33
1.1	Why ALP is Used in Embedded Systems	33
1.2	8051 ALP — Key Instructions	34
2	High-Level Language Programming — C for Embedded	35
2.1	Key C Concepts for Embedded Programming	35
2.2	Processor Directives in Embedded C	35
2.3	Functions and Macros in Embedded C	36
2.4	Embedded C++ Concept	36
3	Real-Time Operating System (RTOS)	37
3.1	OS Overview and Process Management	37
3.2	RTOS Task States	38
3.3	Task Scheduling Algorithms	38
3.4	Comparison of RTOS Scheduling Algorithms	39
3.5	Basic Design Rules Using RTOS	39
Module 5: Introduction to Microchip PIC16 Family		43
1	Introduction to Microchip PIC16 Family	43
1.1	PIC Architecture Classifications	43
1.2	Key Architectural Features of PIC Mid-Range (PIC16)	43
2	PIC16F873 — Architecture and Features	44
2.1	PIC16F873 — Key Specifications	44

3	On-Chip Peripherals of PIC16F873	45
3.1	Watchdog Timer (WDT)	45
3.2	ADC (Analog-to-Digital Converter)	45
3.3	Asynchronous Serial Port — USART	46
3.4	SPI and I2C — MSSP Module	46
4	Interfacing with PIC16F873	46
4.1	LCD Interfacing	46
4.2	ADC Interfacing (Sensors)	46
4.3	Stepper Motor Interfacing	47
4.4	Keyboard (Keypad) Interfacing	47
4.5	DAC (Digital-to-Analog Converter)	47
5	Case Studies of Embedded System Design	47
5.1	Case Study 1: Automated Chocolate Vending Machine	47
5.2	Case Study 2: Digital Camera Controller	48

Module 1

What is an Embedded System?

1. What is an Embedded System?

An embedded system is a combination of hardware and software designed to perform a specific dedicated function within a larger system. Unlike general-purpose computers, embedded systems are purpose-built and optimized for a particular task.

Embedded System — Definition

An **embedded system** is a dedicated computing system that is tightly integrated into the device it controls. It consists of a microprocessor or microcontroller, memory, I/O interfaces, and software (firmware) designed to perform one or more specific tasks — often with real-time constraints and limited resources (power, memory, processing speed).

1.1 Key Characteristics

- **Single-function:** Designed to perform a specific task (e.g., engine control, temperature monitoring).
- **Real-time operation:** Must respond to events within strict time deadlines.
- **Resource-constrained:** Limited memory (RAM/ROM), processing power, and energy budget.
- **Reliability:** Must operate continuously and correctly, often without human intervention.
- **Embedded into a product:** The computing part is hidden inside a larger product (e.g., washing machine, car).

2. Embedded Processor in a System

Embedded Processor

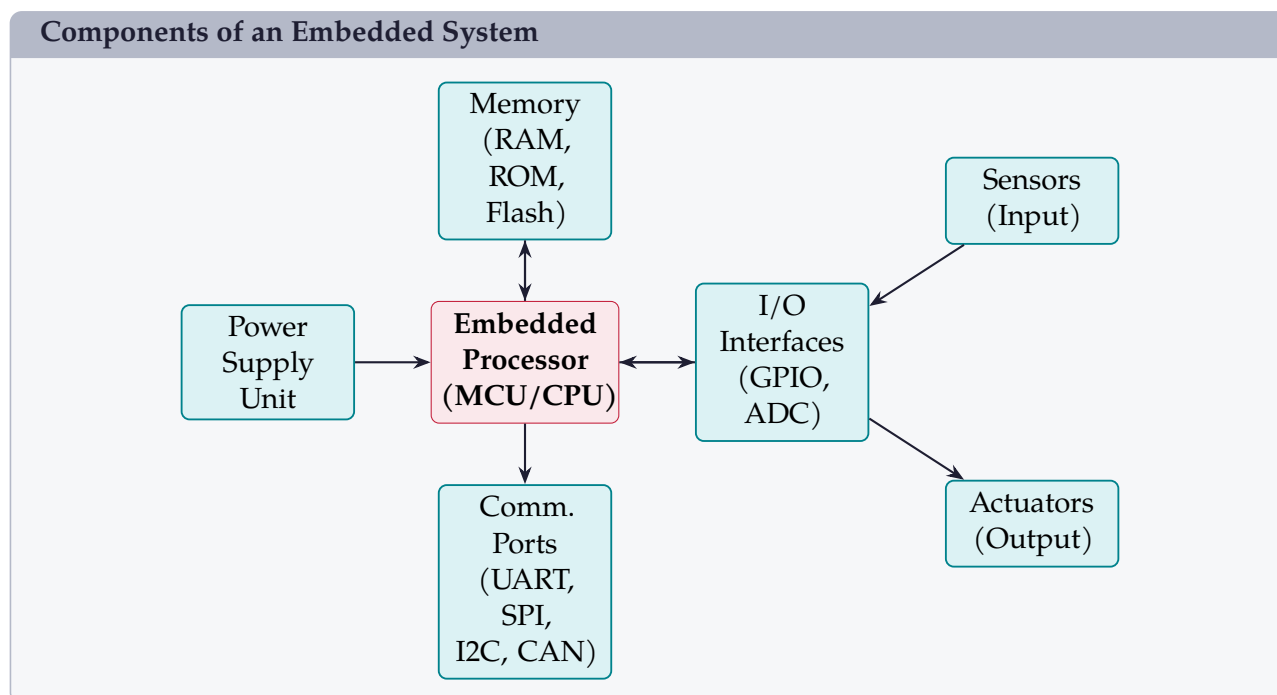
An **embedded processor** is the central computational element of an embedded system. It can be a **microprocessor** (requires external memory and peripherals), a **microcontroller** (integrates CPU, memory, and peripherals on a single chip), a **DSP** (optimized for signal processing), or an **FPGA/ASIC** (for high-speed, custom logic).

2.1 Types of Embedded Processors

Processor Type	Description	Examples / Use
Microcontroller (MCU)	CPU + RAM + ROM + I/O on one chip	8051, PIC, ARM Cortex-M, AVR
Microprocessor (MPU)	CPU only; needs external memory and peripherals	Intel 8086, ARM Cortex-A
DSP Processor	Optimized for multiply-accumulate (MAC) operations	TMS320, SHARC
FPGA / ASIC	Reconfigurable / custom-designed logic fabric	Xilinx, Intel Altera
SoC	Entire system (CPU + GPU + memory + I/O) on one die	Qualcomm Snapdragon, Raspberry Pi SoC

3. Components of an Embedded System

An embedded system is made up of several hardware and software components that work together.



3.1 Hardware Components

- **Processor Core:** The CPU that fetches, decodes, and executes instructions.
- **Memory:**
 - **ROM / Flash:** Stores firmware (non-volatile, permanent program code).
 - **RAM:** Temporary storage for variables and stack (volatile).

- **EEPROM:** Non-volatile, electrically erasable; stores configuration data.
- **I/O Ports:** Digital GPIO, analog ADC/DAC inputs and outputs.
- **Communication Interfaces:** UART, SPI, I2C, CAN, USB.
- **Timers/Counters:** Generate time delays, PWM signals, and measure frequency.
- **Clock Source:** Crystal oscillator, RC oscillator, or PLL — drives system timing.
- **Power Supply:** Regulated DC supply (often 3.3V or 5V); may include battery management.

3.2 Software Components

- **Boot Loader:** First code that runs on power-up; initializes hardware and loads the main program.
- **Operating System / RTOS:** Manages tasks, scheduling, memory, and I/O (optional; may be bare-metal).
- **Device Drivers:** Software layer that abstracts hardware peripherals (UART driver, SPI driver, etc.).
- **Application Software:** The main program implementing the embedded task.
- **Middleware:** Protocol stacks (TCP/IP, USB), file systems, or GUI libraries.

4. Embedded Software in a System

Embedded Software (Firmware)

Embedded software, also called **firmware**, is the software permanently programmed into the non-volatile memory (ROM/Flash) of an embedded system. It directly controls hardware and implements the system's application logic. Unlike desktop software, it cannot be easily updated by the end-user.

4.1 Levels of Embedded Software

The embedded software stack has distinct layers, each serving a different purpose:

Embedded Software Layered Architecture

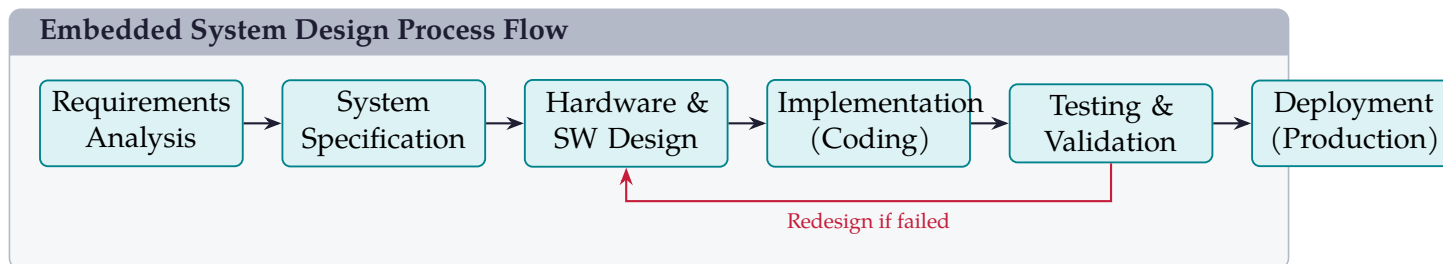
Application Software (Task-specific logic)
Middleware (Protocol stacks, File systems, GUI)
RTOS / Bare-metal Scheduler
Device Drivers (HAL — Hardware Abstraction Layer)
BSP / Boot Loader (Board Support Package)
Hardware (MCU, Memory, Peripherals)

4.2 Brief Introduction to Embedded Software Categories

- **Programming Languages Used:** Assembly language (for low-level hardware control), C (most common for embedded), and C++ (for OOP in embedded, limited use).
- **Bare-metal Programming:** No OS; application code runs directly on hardware. Used in simple, resource-constrained systems.
- **RTOS-based Programming:** Uses a real-time OS for multi-task management. Used when multiple functions must run concurrently with timing guarantees.
- **Firmware Updates (OTA):** Modern embedded systems support Over-The-Air updates to patch firmware without physical access.

5. Design Process in Embedded Systems

The development of an embedded system follows a structured design process to ensure the product meets functional, real-time, power, and cost requirements.



5.1 Design Steps Explained

1. **Requirements Analysis:** Identify what the system must do — functional requirements (what it does), non-functional requirements (speed, power, cost, size, reliability). Example: "Control motor speed based on temperature sensor; respond within 10 ms."
2. **System Specification:** Define architecture — partitioning between hardware and software. Specify processor type, memory size, I/O count, communication protocol, and OS requirements.
3. **Hardware Design:** Schematic design (circuit), PCB layout, selection of MCU/processor, memory, power supply IC, and peripheral components.
4. **Software Design:** Algorithm design, data flow diagrams, state machines. Choosing programming language (C/Assembly), RTOS selection, ISR design.
5. **Implementation:** Writing firmware code (in C or Assembly). Cross-compilation using a host computer to generate binary for the target MCU. Flashing the binary into MCU flash memory.
6. **Testing and Validation:** Unit testing (individual modules), integration testing, hardware-in-the-loop (HIL) testing, real-time performance testing. Verify timing deadlines, power consumption, and EMI compliance.
7. **Deployment:** Mass production, product integration, final firmware version locked, field testing.

5.2 Hardware–Software Co-design

Modern embedded design uses **hardware–software co-design** — HW and SW are developed concurrently and their partitioning is optimized to meet cost and performance targets.

Aspect	Hardware Implementation	Software Implementation
Speed	Very fast (gate-level logic)	Slower (instruction-by-instruction)
Flexibility	Fixed after fabrication	Easily modified (just reflash)
Cost	High NRE (Non-Recurring Engineering)	Low cost for updates
Example	CRC calculation in FPGA	CRC computed in C function

Quick Revision — Module 1

Overview of Embedded Systems | PE-EC703A

Key Formulas at a Glance

System Clock Period	$T = \frac{1}{f_{clk}}$ (e.g., 12 MHz clock $\Rightarrow T = 83.3$ ns)
Instruction Cycle	$t_{inst} = \frac{N_{cycles}}{f_{clk}}$ (N_{cycles} = machine cycles per instruction)
Power Dissipation	$P = C \cdot V_{DD}^2 \cdot f$ (dynamic power in CMOS logic)
Response Time Deadline	$t_{response} \leq t_{deadline}$ (real-time constraint; must always hold)

One-Line Definitions

- **Embedded System:** Dedicated computing system embedded inside a product to perform a specific function with real-time constraints.
- **Firmware:** Software permanently stored in ROM/Flash; directly controls hardware.
- **MCU:** Microcontroller Unit — CPU, RAM, ROM, and peripherals all on one chip.
- **MPU:** Microprocessor Unit — CPU only; needs external support chips.
- **RTOS:** Real-Time Operating System — guarantees task completion within hard time deadlines.
- **HAL:** Hardware Abstraction Layer — software interface that hides hardware details from application code.
- **BSP:** Board Support Package — low-level software that initializes specific hardware board peripherals.
- **Bare-metal:** Programming directly on hardware without an operating system.
- **Cross-compiler:** A compiler running on a host PC that generates code for a different target architecture (e.g., ARM).
- **Hard real-time:** System where missing a deadline causes catastrophic failure (e.g., airbag control).
- **Soft real-time:** System where missing a deadline degrades performance but is not fatal (e.g., video streaming).

Frequently Asked / Viva Questions

Q1. Define an embedded system. How does it differ from a general-purpose computer?

An embedded system is a dedicated computing unit integrated into a product to perform a specific function (e.g., car ABS controller). A

Frequently Asked / Viva Questions (contd.)

general-purpose computer (PC) can run any program. Embedded systems are resource-constrained, single-function, and often real-time.

Q2. What are the key components of an embedded system?

- Processor (MCU/MPU), Memory (RAM/ROM/Flash), I/O Interfaces
- Communication ports (UART, SPI, I2C, CAN), Timers, Clock source, Power supply

Q3. What is the difference between a microprocessor and a microcontroller?

A **microprocessor** contains only the CPU; it needs external RAM, ROM, and peripherals. A **microcontroller** integrates CPU, RAM, ROM, and peripherals on a single chip — lower cost, smaller size, less power.

Q4. What is firmware? Where is it stored?

Firmware is the embedded software (program code) that directly controls hardware. It is stored in **non-volatile memory** — ROM, Flash, or EEPROM — so it persists after power-off.

Q5. What is meant by real-time in embedded systems?

Real-time means the system must complete its tasks within guaranteed time limits (deadlines). In **hard real-time** systems, missing a deadline causes system failure. In **soft real-time**, performance degrades but operation continues.

Q6. List the steps in the embedded system design process.

Requirements analysis → System specification → HW & SW design → Implementation (coding) → Testing and validation → Deployment.

Q7. What is hardware–software co-design?

It is the simultaneous design of hardware and software components, optimizing their partitioning to meet cost, performance, and power goals. Functions can be implemented in either HW (faster, fixed) or SW (flexible, cheaper to modify).

Q8. What is a BSP (Board Support Package)?

A BSP is a collection of low-level software routines specific to a hardware board. It initializes the processor, memory, clocks, and peripherals during boot, providing a foundation for higher-level software layers.

Q9. What is a cross-compiler and why is it needed?

A cross-compiler runs on a **host machine** (e.g., x86 PC) and generates executable code for a **target architecture** (e.g., ARM, 8051). It is needed because embedded targets often lack the resources to run a compiler themselves.

Q10. Give three examples of embedded systems in daily life.

Washing machine controller, automobile engine control unit (ECU), digital camera, ATM machine, and smart thermostat.

Frequently Asked / Viva Questions (contd.)

Q11. What is the role of a bootloader in an embedded system?

The bootloader is the first code executed after power-on. It initializes hardware (clocks, memory), and loads the main application firmware from flash into RAM (if needed) and transfers execution control to the application.

Q12. What is meant by a “resource-constrained” embedded system?

Resource-constrained means the system has very limited RAM (e.g., 256 bytes to 64 KB), flash (8 KB to 1 MB), processing power (8-bit, 8 MHz), and battery capacity. Firmware must be optimized to work within these tight constraints.

Comparison: Microprocessor vs. Microcontroller vs. DSP

Feature	Microprocessor	Microcontroller	DSP Processor
Integration	CPU only	CPU + RAM + ROM + I/O	CPU + MAC unit
External Parts	Many needed	Few or none	Some needed
Power	High	Low	Medium
Cost	High (system level)	Low (single chip)	Medium–High
Best For	Complex OS-based apps	Dedicated control tasks	Signal processing (audio, radar)
Example	Intel Core i7	PIC16F877, STM32	TI TMS320C6748

Module 2

8051 Microcontroller — Architecture Overview

1. 8051 Microcontroller — Architecture Overview

The Intel 8051 is an 8-bit microcontroller introduced in 1980. It is one of the most widely taught architectures in embedded systems education due to its simplicity and self-contained design.

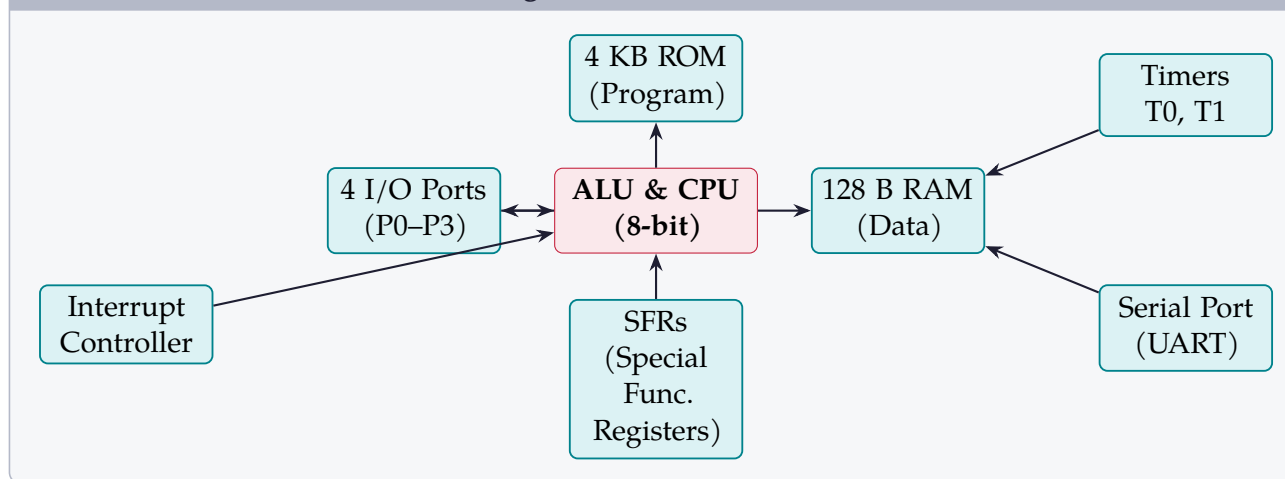
8051 Microcontroller

The **8051** is an 8-bit, single-chip microcontroller with on-chip program memory (ROM), data memory (RAM), I/O ports, timers, a serial port, and interrupt logic. It operates on a Harvard architecture (separate program and data memory buses).

1.1 Key Features of 8051

- **CPU:** 8-bit ALU, accumulator (A), B register, PSW (Program Status Word).
- **ROM (Program Memory):** 4 KB on-chip; expandable to 64 KB externally.
- **RAM (Data Memory):** 128 bytes on-chip; upper 128 bytes are SFRs (Special Function Registers).
- **I/O Ports:** Four 8-bit bidirectional I/O ports (P0, P1, P2, P3).
- **Timers:** Two 16-bit timers/counters (Timer 0 and Timer 1).
- **Serial Port (UART):** Full-duplex serial communication (TXD/RXD on P3).
- **Interrupts:** Five interrupt sources — INT0, INT1, Timer0, Timer1, Serial.
- **Clock:** Typically 12 MHz crystal; one machine cycle = 12 clock cycles.

8051 Internal Architecture Block Diagram



1.2 Real-World Interfacing of 8051

- **LED / Switch:** GPIO pins directly drive LEDs (with current-limiting resistors); switches pulled up to VCC.
- **LCD (16x2):** Driven via P2 (data bus) and P3 control lines (RS, RW, EN).
- **ADC (e.g., ADC0804):** External ADC connected via parallel data bus and control signals.
- **Stepper Motor:** Driven via ULN2003 driver IC connected to P1.
- **Serial Communication:** TXD (P3.1) and RXD (P3.0) connect to MAX232 (RS-232 level converter) for PC communication.

2. ARM Processor Architecture

ARM (Advanced RISC Machine) Processor

ARM is a family of RISC (Reduced Instruction Set Computer) processor architectures developed by ARM Holdings. ARM processors are characterized by a 32-bit (or 64-bit) load-store architecture, fixed-length 32-bit instructions (Thumb: 16-bit compressed), low power consumption, and high code density.

2.1 ARM vs. 8051 — Key Differences

Feature	8051	ARM Cortex-M
Architecture	CISC (8-bit, Harvard)	RISC (32-bit, Harvard)
Registers	A, B, R0–R7 (bank-switched)	16 general-purpose 32-bit registers (R0–R15)
Clock Speed	12 MHz (typical)	Up to 168 MHz+ (Cortex-M4)
Pipeline	Single-stage	3-stage (Cortex-M0) to 6-stage (Cortex-M4)
Instruction Set	Complex, variable-length	RISC + Thumb-2 (16/32-bit)
Built-in DSP	No	Yes (Cortex-M4 has FPU + DSP extensions)
Power	Low (simple core)	Ultra-low to medium
Example Chips	AT89S52, DS89C4x0	STM32, LPC1768, Nordic nRF52

2.2 ARM Processor Features

- **RISC Pipeline:** Multi-stage instruction pipeline (Fetch → Decode → Execute) allows higher throughput.
- **Load-Store Architecture:** Only Load and Store instructions access memory; all other operations work on registers.
- **Thumb Instruction Set:** 16-bit compressed instructions reduce code size by ~30% while maintaining most of the 32-bit performance.
- **Barrel Shifter:** Integrated into ALU — allows free-of-cost shifts/rotates within any data-processing instruction.
- **NVIC (Nested Vectored Interrupt Controller):** Fast, low-latency interrupt handling with configurable priority levels.
- **Cortex-M Variants:** M0 (ultra-low power), M3 (general purpose), M4 (with FPU and DSP), M7 (high performance).

3. Processor and Memory Organization

3.1 Memory Types in Embedded Systems

Memory Type	Characteristics	Use in Embedded
ROM (Mask ROM)	Non-volatile, factory-programmed, read-only	Fixed firmware in mass-produced ICs
Flash	Non-volatile, electrically erasable, page-by-page write	Main program storage in modern MCUs
EEPROM	Non-volatile, byte-wise electrically erasable	Configuration data, calibration parameters
SRAM	Volatile, fast, no refresh needed	CPU data memory, stack, heap
DRAM	Volatile, needs refresh, high density	External memory for complex embedded systems (not MCU)
SDRAM	Synchronous DRAM, higher speed	External RAM for embedded Linux (e.g., Raspberry Pi)

3.2 Harvard vs. Von Neumann Architecture

Feature	Harvard Architecture	Von Neumann Architecture
Memory Buses	Separate instruction and data buses	Single shared bus
Simultaneous Access	Yes — fetch instruction while reading data	No — only one at a time
Speed	Faster (no bus contention)	Slower (bus bottleneck)
Examples	8051, PIC, ARM Cortex-M	Intel x86 PC
Complexity	Higher (dual bus hardware)	Simpler

3.3 Memory Map of a Typical MCU

The memory-mapped addressing scheme of a microcontroller assigns fixed addresses to each memory region:

Typical MCU Memory Map (ARM Cortex-M)

Peripherals (0x40000000–)	0x4000 0000
SRAM (0x20000000–)	0x2000 0000
Flash / Code (0x08000000–)	0x0800 0000
Boot ROM / System Memory	0x1FFF 0000
Vector Table + Option Bytes	0x0000 0000
0x00000000 (Alias region)	

4. Parallelism in Instruction Level

Instruction-Level Parallelism (ILP)

Instruction-Level Parallelism (ILP) is the ability of a processor to execute multiple instructions simultaneously, overlapping their execution stages using techniques such as **pipelining**, **superscalar execution**, and **VLIW** (Very Long Instruction Word).

4.1 Pipelining

Pipelining divides instruction execution into **stages**, so that multiple instructions can be in different stages of execution simultaneously — like an assembly line.

3-Stage Instruction Pipeline (ARM Cortex-M0)

Cycle	1	2	3	4	5
Instr 1	Fetch	Decode	Execute		
Instr 2		Fetch	Decode	Execute	
Instr 3			Fetch	Decode	Execute

4.2 Pipeline Hazards

Hazard Type	Cause	Solution
Data Hazard	Instruction depends on result of a previous instruction still in pipeline	Register forwarding (bypassing), stall/bubble insertion
Control Hazard	Branch instruction changes program flow before pipeline completes	Branch prediction, pipeline flush
Structural Hazard	Two instructions need same hardware resource simultaneously	Resource duplication, stall

5. Processor and Memory Selection

Selecting the right processor and memory for an embedded application requires balancing multiple constraints:

Processor Selection Criteria

- **Processing Speed:** Clock frequency (MHz) and MIPS (Million Instructions Per Second) rating.
- **Word Width:** 8-bit (simple, low-cost) vs. 16-bit vs. 32-bit (high performance).
- **On-chip Peripherals:** Required ADC channels, communication ports, timers.
- **Power Consumption:** Active current (I_{DD}), sleep current ($I_{standby}$) — critical for battery-powered devices.
- **Cost:** Unit price must fit product BOM (Bill of Materials) budget.
- **Toolchain and Ecosystem:** Availability of compilers, debuggers, libraries, and community support.
- **Temperature Range:** Industrial (-40°C to $+85^{\circ}\text{C}$), automotive (-40°C to $+125^{\circ}\text{C}$).

5.1 Memory Selection Criteria

- **Volatile vs. Non-volatile:** Program code needs non-volatile (Flash); variables need volatile (SRAM).
- **Capacity:** Must accommodate program code, constants, variables, stack, and heap with margin.
- **Access Speed:** RAM access time must be compatible with processor clock (no wait states needed).
- **Endurance:** Flash has limited write cycles ($\sim 100,000$ erase/write cycles) — critical for OTA update designs.
- **Interface:** Parallel bus (faster) vs. serial (SPI/I2C Flash — slower but fewer pins).

Quick Revision — Module 2

Embedded Hardware: Processor & Memory | PE-EC703A

Key Formulas at a Glance

Machine Cycle Time (8051)	$t_{mc} = \frac{12}{f_{osc}}$ (12 oscillator periods per machine cycle)
Processing Speed	$MIPS = \frac{f_{clk}}{CPI \times 10^6}$ (CPI = Cycles Per Instruction)
Pipeline Speedup	$S = \frac{t_{no-pipeline}}{t_{pipeline}} \approx N$ (N = number of stages, ideal)
Memory Address Space	Addressable locations = 2^n (n = address bus width in bits)

One-Line Definitions

- **8051:** An 8-bit Harvard-architecture microcontroller with 4 KB ROM, 128 B RAM, 4 I/O ports, and 2 timers.
- **ARM:** Advanced RISC Machine; 32-bit low-power RISC processor widely used in embedded and mobile devices.
- **RISC:** Reduced Instruction Set Computer — few, simple, fixed-length instructions; high clock rate.
- **Harvard Architecture:** Separate buses for program memory and data memory allowing simultaneous access.
- **Von Neumann:** Single shared bus for instructions and data; simpler but slower.
- **Pipeline:** Technique to overlap execution of multiple instructions by dividing into fetch-decode-execute stages.
- **ILP:** Instruction-Level Parallelism — executing multiple instructions in the same clock cycle.
- **Flash Memory:** Non-volatile, electrically erasable memory used to store program code in MCUs.
- **SRAM:** Static RAM — volatile, fast, no refresh; used for data memory and stack in MCUs.
- **SFR:** Special Function Register in 8051 (addresses 80H–FFH) controlling timers, serial port, I/O, interrupts.
- **Thumb Instruction Set:** 16-bit compressed ARM instruction set reducing code size by ~30%.
- **NVIC:** Nested Vectored Interrupt Controller in ARM; handles priority-based, low-latency interrupts.

Frequently Asked / Viva Questions**Q1. What is the architecture of the 8051 microcontroller?**

The 8051 is an 8-bit Harvard-architecture MCU with a 4 KB ROM (program), 128 bytes RAM (data), four 8-bit I/O ports, two 16-bit timers, one UART serial port, and five interrupt sources. One machine cycle = 12 oscillator clock cycles.

Q2. What is the difference between Harvard and Von Neumann architecture?

Harvard uses separate buses for program and data memory, enabling simultaneous fetch and data access (faster). Von Neumann uses a single bus for both (simpler but has bottleneck). 8051 and ARM MCUs use Harvard; x86 PCs use Von Neumann.

Q3. What is pipelining? What is its advantage?

Pipelining divides instruction execution into stages (Fetch, Decode, Execute) and processes multiple instructions concurrently in different stages. Advantage: Increases throughput (instructions completed per clock cycle) without increasing clock frequency.

Q4. Name the three types of pipeline hazards.

- **Data Hazard:** Dependency between instructions (solved by forwarding/stall).
- **Control Hazard:** Branch instructions disrupting pipeline flow (solved by branch prediction).
- **Structural Hazard:** Two instructions compete for same hardware resource.

Q5. Why is ARM preferred over 8051 for modern embedded systems?

ARM (Cortex-M) offers 32-bit data width, higher clock speeds (up to 168 MHz), a pipelined RISC core, Thumb instruction set for code efficiency, built-in NVIC, optional FPU/DSP, and ultra-low power modes. 8051 is only 8-bit and runs at 12 MHz.

Q6. What is the Thumb instruction set in ARM?

Thumb is a 16-bit compressed version of the 32-bit ARM instruction set. It reduces code size by ~30% while retaining most of the performance, making it ideal for memory-constrained embedded systems.

Q7. What are Special Function Registers (SFRs) in 8051?

SFRs are memory-mapped registers (addresses 80H–FFH) that control the MCU's internal peripherals such as ports (P0–P3), timers (TCON, TMOD), serial port (SCON, SBUF), interrupt control (IE, IP), and the stack pointer (SP).

Q8. What is the difference between Flash memory and EEPROM?

Flash memory is erased in large **blocks/pages** and is much faster and higher-density — used for program code storage. EEPROM is erased byte-by-byte (or word-by-word), has fewer erase cycles, but allows fine-grained updates — used for configuration data.

Frequently Asked / Viva Questions (contd.)

Q9. What factors are considered in processor selection for an embedded system?

Processing speed (MIPS), word width (8/16/32-bit), on-chip peripheral set, power consumption, operating temperature range, unit cost, and availability of compilers, development tools, and community support.

Q10. What is memory-mapped I/O?

In memory-mapped I/O, peripheral registers (I/O ports, timers, UART) are assigned addresses in the same address space as RAM. The processor accesses them using regular memory read/write instructions — no separate I/O instructions needed (used in ARM Cortex-M).

Q11. What is the significance of the barrel shifter in ARM?

The barrel shifter in ARM's ALU allows any data-processing instruction to optionally shift or rotate one of its source operands by any amount **in the same clock cycle** as the main operation — effectively providing free shift operations.

Q12. What is MIPS and how is it calculated?

MIPS (Million Instructions Per Second) measures processor performance. $MIPS = f_{clk} / (CPI \times 10^6)$ where CPI is Cycles Per Instruction. For 8051 at 12 MHz with CPI = 12: $MIPS = 12 / (12) = 1$ MIPS.

8051 vs. ARM Cortex-M — At a Glance

Parameter	8051	ARM Cortex-M4
Data Width	8-bit	32-bit
Max Clock	12 MHz (standard)	168 MHz
Architecture	Harvard, CISC-like	Harvard, RISC
Pipeline	None	6-stage
On-chip Flash	4 KB (original)	Up to 2 MB
On-chip SRAM	128 bytes	Up to 256 KB
FPU	No	Yes (single precision)
Interrupts	5 sources	Up to 240 (NVIC)

Module 3

Serial and Parallel Communication Ports

1. Serial and Parallel Communication Ports

Serial vs. Parallel Communication

Serial Communication: Data is transmitted one bit at a time over a single wire (or differential pair). Slower per line but uses fewer wires, suitable for long-distance communication.

Parallel Communication: Multiple bits (e.g., 8 bits) are transmitted simultaneously over multiple wires. Faster but requires more wires; limited to short distances due to skew and crosstalk.

Feature	Serial Port (UART)	Parallel Port
Wires	2 (TXD, RXD) minimum	8+ data lines + control
Speed	Lower (per port)	Higher
Distance	Long range (RS-232, RS-485)	Short (PCB-level, <1 m)
Cost	Low (fewer pins, simpler hardware)	Higher
Examples	UART, SPI, I2C, USB	8051 P0–P3 ports, old printer port

1.1 UART (Universal Asynchronous Receiver/Transmitter)

- **Asynchronous:** No shared clock line — both devices agree on **baud rate** beforehand.
- **Frame format:** Start bit (1, low) + Data bits (5–9) + Optional Parity bit + Stop bit(s) (1 or 2, high).
- **Baud rate:** Number of signal changes per second (symbols/second). At 9600 baud, 1 bit = 104 μ s.
- **Wiring:** TXD of device A connects to RXD of device B, and vice versa. Ground is shared.

UART Baud Rate Formula (8051)

$$\text{Baud Rate} = \frac{f_{osc}}{32 \times 12 \times (256 - \text{TH1})} \quad (\text{Mode 1 or 3, Timer 1 in 8-bit auto-reload})$$

For 9600 baud at $f_{osc} = 11.0592 \text{ MHz}$: $\text{TH1} = \text{FDH} (253)$

2. Timer and Counting Devices

Timer / Counter

A **timer** is a hardware counter that increments on each internal clock pulse (counting time). A **counter** increments on each external event pulse (e.g., counting RPM or pulses). Both use the same hardware register; the source of the clock selects timer vs. counter mode.

2.1 8051 Timer Modes

The 8051 has two timers (Timer 0 and Timer 1), each configurable in 4 modes via TMOD register:

Mode	Description	Use
Mode 0	13-bit timer/counter (TH + 5 LSBs of TL)	Legacy 8048 compatibility
Mode 1	16-bit timer/counter (TH:TL = 65536 counts)	Long time delays
Mode 2	8-bit auto-reload (TL counts, TH is reload value)	Baud rate generation
Mode 3	Two independent 8-bit timers (Timer 0 only)	Dual-purpose timing

Timer Delay Calculation (8051, Mode 1)

$$\text{Time delay} = (65536 - \text{Initial Count}) \times \frac{12}{f_{osc}}$$

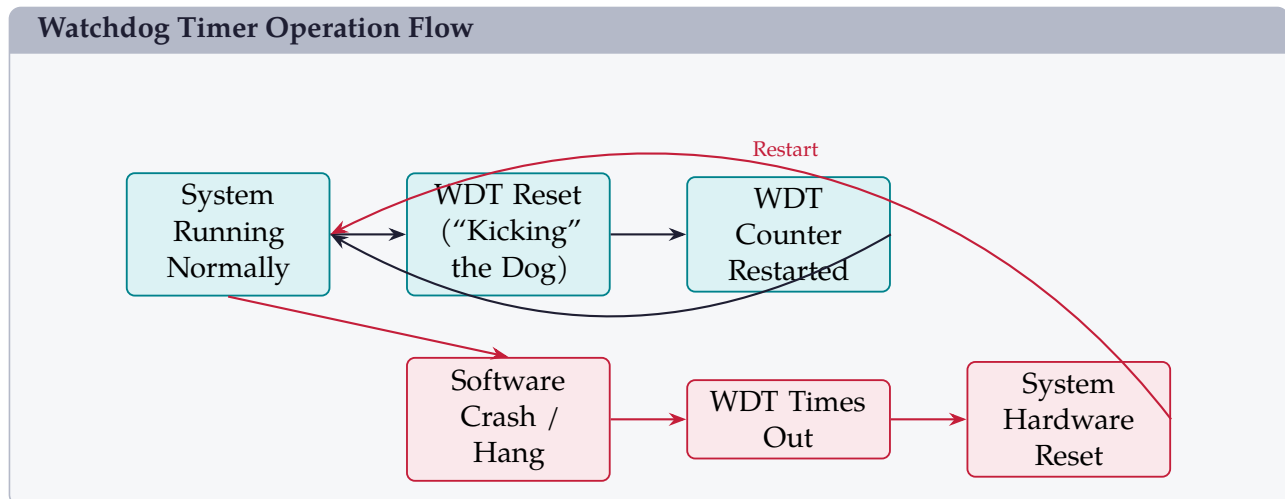
Example: For 10 ms delay at 12 MHz: Initial Count = $65536 - 10000 = 55536 = \text{D8F0H}$

3. Watchdog Timer

Watchdog Timer (WDT)

A **watchdog timer** is a hardware timer that automatically **resets the microcontroller** if the software fails to periodically reset (“kick” or “pet”) it within a defined time interval. It is used to detect and recover from software deadlocks, infinite loops, or crashes in embedded systems.

3.1 Watchdog Timer Operation



- **Kicking (petting) the WDT:** Software writes a specific value to the WDT register before the timeout period expires. This restarts the countdown.
- **WDT Timeout Period:** Configurable (e.g., 100 ms to several seconds). If not reset in this time, the WDT asserts a hardware reset signal.
- **Window Watchdog (WWDT):** Advanced variant — must be kicked *within* a specific time window (not too early, not too late) to detect stuck loops that still kick the WDT.

4. Real-Time Clock (RTC)

Real-Time Clock (RTC)

An **RTC** is a dedicated counter/timer circuit that tracks the current time (hours, minutes, seconds) and date (day, month, year) continuously — even when the main system is powered off — using a separate battery (e.g., CR2032 coin cell) and a 32.768 kHz crystal.

- **Crystal Frequency:** 32.768 kHz = 2^{15} Hz. A 15-bit counter overflows exactly once per second.
- **Interface:** Typically I2C or SPI serial interface (e.g., DS3231, PCF8563).
- **Applications:** Data loggers, digital clocks, alarm systems, access control systems.
- **Alarms/Interrupts:** RTC can generate periodic interrupts (every second, minute, etc.) to wake up the MCU from sleep mode.

5. Serial Bus Communication Protocols

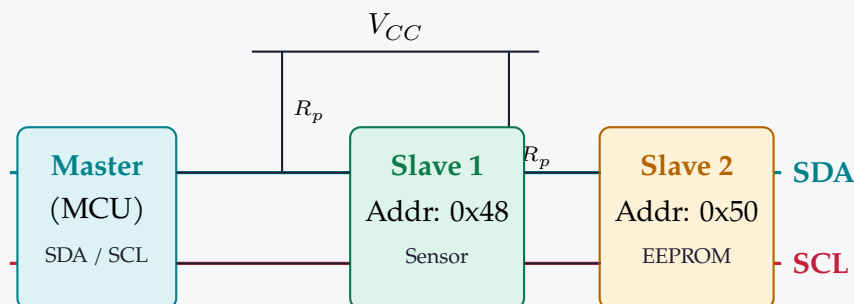
5.1 I2C (Inter-Integrated Circuit) Protocol

I2C Protocol

I2C (pronounced “I-squared-C”) is a synchronous, multi-master, multi-slave, half-duplex serial protocol using only **two wires**: SDA (Serial Data) and SCL (Serial Clock). Developed by Philips (NXP). Each device has a unique 7-bit (or 10-bit) address.

- **Wires:** SDA (bidirectional data) + SCL (clock generated by master). Both are open-drain with pull-up resistors.
- **Speed Modes:** Standard (100 kbps), Fast (400 kbps), Fast-Plus (1 Mbps), High-Speed (3.4 Mbps).
- **Start/Stop Condition:** Start = SDA falls while SCL is high. Stop = SDA rises while SCL is high.
- **ACK/NACK:** After each 8-bit byte, receiver pulls SDA low (ACK) or leaves it high (NACK).
- **Applications:** Connecting sensors (temperature, pressure), EEPROMs, RTC modules, LCD controllers.

I2C Bus — Master–Slave Connection



5.2 CAN (Controller Area Network) Protocol

CAN Bus Protocol

CAN is a robust, multi-master serial bus protocol designed specifically for **automotive and industrial** applications where reliability in high-noise environments is critical. It uses differential signaling on two wires: **CAN-H** (High) and **CAN-L** (Low). No master/slave — all nodes are equal.

- **Differential Signaling:** CAN-H and CAN-L carry inverted signals. Noise affects both equally and is cancelled at the receiver, giving high EMI immunity.
- **Speed:** Up to 1 Mbps (standard CAN), up to 5–8 Mbps (CAN-FD).
- **Message-Based:** No addresses. Each message has an **Identifier (ID)** (11-bit or 29-bit). All nodes receive all messages and filter by ID.
- **CSMA/CA:** Carrier Sense Multiple Access with Collision **Avoidance** — lower ID (higher priority) wins arbitration without destroying the message.

- **Error Detection:** CRC checking, bit stuffing, ACK slots — very robust error handling.
- **Applications:** Automotive ECUs, industrial PLCs, medical devices, robotics.

5.3 ISA (Industry Standard Architecture) — Parallel Bus

ISA Bus

ISA is a **parallel communication standard** developed by IBM (1981) for connecting peripheral cards (sound cards, modems, I/O cards) to the PC motherboard. It uses an 8-bit or 16-bit wide data bus at 8 MHz. Largely obsolete (replaced by PCI, PCIe) but historically significant.

Feature	I2C	CAN	ISA
Type	Serial (synchronous)	Serial (async, differential)	Parallel
Wires	2 (SDA, SCL)	2 (CAN-H, CAN-L)	8/16 data + 20 address + control
Max Speed	3.4 Mbps	1 Mbps (8 Mbps CAN-FD)	8 MHz (16 MB/s)
Distance	Short (PCB-level, <1 m)	Up to 1 km at 50 kbps	Very short (backplane)
Use	Sensors, EEPROMs	Automotive, industrial	Legacy PC expansion cards

6. Interrupt Service Mechanism

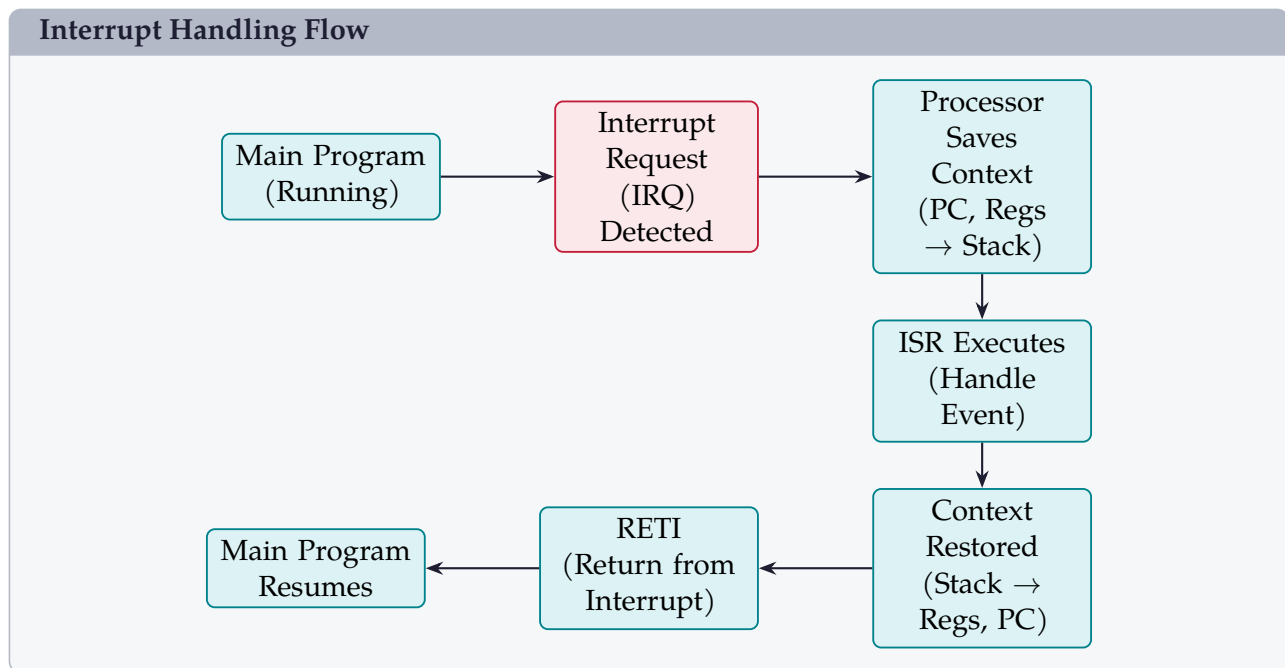
Interrupt

An **interrupt** is a signal sent to the processor by hardware or software indicating that an event requiring immediate attention has occurred. The processor **suspends** its current task, saves its state (PC, registers), and executes a special routine called the **Interrupt Service Routine (ISR)**. After the ISR completes, execution resumes from where it was interrupted.

6.1 ISR (Interrupt Service Routine)

ISR — Interrupt Service Routine

An **ISR** is a software function that is automatically called by the processor when a specific interrupt occurs. It handles the interrupt event (e.g., reads a received byte from UART, clears a timer flag) and must be kept **short and fast** to minimize system latency.



6.2 8051 Interrupt Sources

The 8051 has **5 interrupt sources** with a fixed priority:

Interrupt	Source	Vector Address	Priority (Default)
INT0	External Pin P3.2	0003H	Highest (1)
Timer 0	Timer 0 overflow	000BH	2
INT1	External Pin P3.3	0013H	3
Timer 1	Timer 1 overflow	001BH	4
Serial	UART TI or RI flag	0023H	Lowest (5)

6.3 Different Interrupt Sources (General)

- **External Interrupts:** Triggered by a signal change on a pin (edge-triggered or level-triggered).
- **Timer Interrupts:** Triggered when a timer/counter register overflows.
- **Communication Interrupts:** Triggered when UART receives a byte, or SPI transfer completes.
- **ADC Interrupts:** Triggered when analog-to-digital conversion is complete.
- **Software Interrupts (SWI):** Triggered by executing a special instruction (used in ARM for system calls).
- **Reset:** Non-maskable; forces processor to restart (highest priority).

7. Multiple Interrupts, Interrupt Latency and Deadline

7.1 Multiple Interrupt Handling

When several interrupts occur simultaneously, the system must resolve priority:

- **Fixed Priority:** Each interrupt has a fixed, pre-assigned priority level. Higher priority ISR always preempts lower priority (8051 default behavior via IP register).
- **Nested Interrupts:** A higher-priority interrupt can interrupt a currently-executing lower-priority ISR. Requires saving the lower ISR's state on the stack.
- **Interrupt Masking:** Individual interrupts can be enabled/disabled via IE (Interrupt Enable) register bits to prevent interference during critical code sections.

8051 Interrupt Enable (IE) Register

EA	—	—	ES	ET1	EX1	ET0	EX0
Bit 7	6	5	4	3	2	1	0

EA = Global enable; ES = Serial; ET1 = Timer1; EX1 = INT1; ET0 = Timer0; EX0 = INT0

7.2 Interrupt Latency

Interrupt Latency

Interrupt latency is the time elapsed from when an interrupt signal is asserted to when the first instruction of the ISR begins executing. It includes: detecting the interrupt, completing the current instruction, saving context (pushing to stack), and fetching the ISR vector.

Interrupt Latency Components

$$t_{\text{latency}} = t_{\text{detect}} + t_{\text{inst_complete}} + t_{\text{context_save}} + t_{\text{vector_fetch}}$$

- t_{detect} : Time for hardware to sample and recognize the interrupt signal.
- $t_{\text{inst_complete}}$: Time to finish the currently executing instruction (worst case: longest instruction).
- $t_{\text{context_save}}$: Time to push PC (and registers, if applicable) to stack.
- $t_{\text{vector_fetch}}$: Time to read the ISR address from the vector table and jump.

ARM Cortex-M: Typical latency = 12 clock cycles (with zero wait-state memory).

7.3 Interrupt Deadline

Interrupt Deadline

An **interrupt deadline** is the maximum allowable time window within which an interrupt request must be fully serviced. It starts at the moment the interrupt event occurs (trigger) and ends when the required response action is completely finished.

7.3.1 Classification of Interrupt Deadlines

- **Hard Deadline:** Missing the deadline results in total system failure, physical damage, or safety hazards (e.g., airbag deployment, pacemaker pacing pulse, anti-lock braking systems).
- **Soft Deadline:** Missing the deadline degrades the quality of service (QoS) but does not cause system failure (e.g., dropping a frame in video playback, delaying a UI button highlight).
- **Firm Deadline:** Missing the deadline means the processed data is completely useless, but it does not cause catastrophic damage (e.g., packet arrival in telecommunication where outdated packets are simply discarded).

Real-Time Constraint & Timing Budget

For a system to be real-time compliant, the sum of interrupt latency and the ISR execution time must not exceed the deadline:

$$t_{\text{latency}} + t_{\text{ISR_execution}} \leq t_{\text{deadline}}$$

Where:

- t_{latency} : The time taken to detect the interrupt, save context, and jump to the ISR start.
- $t_{\text{ISR_execution}}$: The execution time of the active instructions inside the ISR.
- t_{deadline} : The timing budget imposed by the physical process/application.

7.3.2 Key Factors Causing Deadline Violations

1. **Interrupt Nesting & Blocking:** If a higher-priority interrupt preempts the current ISR, or if interrupts are globally disabled (during critical sections of the OS), the execution of the lower-priority ISR is delayed, increasing its response time.
2. **Interrupt Jitter:** The variance in interrupt latency caused by processor states (pipeline stalls, cache misses, branch mispredictions, or instruction execution state).
3. **Over-complex ISRs:** Performing heavy math (e.g., floating-point computations, FFT) or blocking calls (e.g., waiting for serial I/O or delays) inside the ISR.

7.3.3 Design Strategies to Meet Deadlines

- **Keep ISRs “Short and Sweet”:** Perform only the bare minimum operations (e.g., reading a register, clearing a flag, placing data in a circular buffer) and signal the main loop or a background task using a volatile flag or RTOS semaphore.
- **Deferred Processing (Bottom Half):** Defer the heavy non-critical execution to the background (often called a deferred procedure call, tasklet, or thread).
- **Proper Priority Assignment:** Use **Deadline Monotonic Scheduling (DMS)**, where interrupts with shorter deadlines are assigned higher priority, regardless of their arrival rate.

Quick Revision — Module 3

I/O Types & Interrupt Service Mechanism | PE-EC703A

Key Formulas at a Glance

UART Baud Rate (8051)	$\text{Baud} = \frac{f_{osc}}{32 \times 12 \times (256 - \text{TH1})}$
Timer Delay (Mode 1)	$t_{delay} = (65536 - \text{Init_Count}) \times \frac{12}{f_{osc}}$
Interrupt Deadline	$t_{latency} + t_{ISR} \leq t_{deadline}$
RTC Crystal Freq	$f = 32768 \text{ Hz} = 2^{15} \text{ Hz} \quad (15\text{-bit counter, 1 overflow/second})$

One-Line Definitions

- **UART:** Asynchronous serial protocol using start/stop bits; configurable baud rate.
- **I2C:** 2-wire synchronous serial bus (SDA + SCL); multi-master, 7-bit addressing.
- **CAN:** 2-wire differential serial bus for automotive/industrial; message-ID arbitration.
- **ISA:** Legacy 8/16-bit parallel expansion bus from IBM PC (1981); obsolete.
- **Timer:** Hardware counter that counts internal clock pulses to measure time.
- **Counter:** Hardware timer counting external event pulses.
- **WDT:** Watchdog Timer — resets MCU if software fails to kick it periodically.
- **RTC:** Real-Time Clock — tracks date and time using 32.768 kHz crystal and coin-cell battery.
- **Interrupt:** Hardware/software signal that suspends main program to run ISR.
- **ISR:** Interrupt Service Routine — short function handling the interrupt event.
- **Interrupt Latency:** Delay from interrupt request to first ISR instruction.
- **Interrupt Deadline:** Maximum allowable time for ISR to complete its response.
- **NVIC:** ARM's Nested Vectored Interrupt Controller; handles up to 240 priority-based interrupts.

Frequently Asked / Viva Questions

Q1. What is the difference between serial and parallel communication?

Serial sends data one bit at a time (fewer wires, longer distance). Parallel sends multiple bits simultaneously (more wires, faster but short-distance only).

Q2. What is UART? Describe its frame format.

UART is Universal Asynchronous Receiver/Transmitter. Frame: **Start bit** (1 bit,

Frequently Asked / Viva Questions (contd.)

low) + **Data** (5–9 bits, LSB first) + **Parity bit** (optional) + **Stop bit(s)** (1 or 2 bits, high). Both ends agree on baud rate; no clock wire.

Q3. What are the two wires in I2C and what do they do?

SDA (Serial Data Line): carries data bidirectionally. **SCL** (Serial Clock Line): carries the synchronization clock generated by the master. Both are open-drain with pull-up resistors.

Q4. Why is CAN used in automobiles instead of I2C or UART?

CAN uses differential signaling which is immune to electromagnetic noise. It supports multi-master arbitration (priority-based, non-destructive), reliable error detection (CRC, ACK, bit stuffing), and distances up to 1 km at 50 kbps — all essential in a noisy vehicle environment.

Q5. What is a Watchdog Timer? Why is it important?

A WDT is a hardware timer that resets the MCU if software fails to periodically reset it. It is critical in embedded systems because it provides autonomous recovery from software hangs, infinite loops, or crashes — without human intervention.

Q6. What is the purpose of the 32.768 kHz crystal in an RTC?

32.768 kHz = 2^{15} Hz. A 15-bit counter driven at this frequency overflows exactly once every second, providing a precise 1-second tick for timekeeping without complex division logic.

Q7. Define interrupt latency and list its components.

Interrupt latency is the time from interrupt assertion to ISR start. Components: interrupt detection time + current instruction completion time + context save time (PC/regs to stack) + vector table fetch time.

Q8. What is a vector table in embedded systems?

A vector table is a fixed memory location (e.g., 0x00000000 in ARM) that stores the starting addresses (vectors) of all ISRs. When an interrupt occurs, the CPU reads the corresponding vector and jumps to the ISR.

Q9. How does the 8051 handle two simultaneous interrupts?

The 8051 uses a fixed priority scheme. INT0 has the highest priority and Serial has the lowest. If two interrupts occur simultaneously, the higher-priority one is serviced first. Priority can also be adjusted using the IP (Interrupt Priority) register to create two priority levels.

Q10. What is the difference between edge-triggered and level-triggered interrupts?

Edge-triggered: ISR fires on signal transition (rising or falling edge). Responds to event occurrence. **Level-triggered:** ISR fires as long as the signal remains at the active level. The ISR keeps being called until the condition is cleared.

Q11. Why should an ISR be kept as short as possible?

Frequently Asked / Viva Questions (contd.)

Long ISRs increase interrupt latency for other interrupts, can miss deadlines, may cause data loss (e.g., UART buffer overflow), and increase system unpredictability. Heavy processing should be deferred to the main loop using a flag set by the ISR.

Q12. What is the Timer Mode 2 in 8051 and why is it used for baud rate generation?

Mode 2 is an 8-bit auto-reload mode where TL counts from the reload value (stored in TH) to 255 and overflows. On overflow, TL is automatically reloaded from TH. This creates a precise, continuous periodic interrupt — ideal for generating UART baud rates without software intervention.

Serial Communication Protocols at a Glance

Protocol	Wires	Speed	Masters	Typical Use
UART	2 (TX, RX)	115.2 kbps typical	1	Debug console, GPS
SPI	4 (MOSI, MISO, SCK, CS)	Up to 50 Mbps	1	Flash memory, LCD
I2C	2 (SDA, SCL)	Up to 3.4 Mbps	Multiple	Sensors, EEPROM
CAN	2 (CAN-H, CAN-L)	Up to 1 Mbps	Multi-master	Automotive ECU
USB	4 (D+, D-, VCC, GND)	Up to 10 Gbps	1 host	PC peripherals

Module 4

Assembly Language Programming (ALP) in Embedded Systems

1. Assembly Language Programming (ALP) in Embedded Systems

Assembly Language Programming (ALP)

Assembly language is a low-level programming language that uses **mnemonics** (symbolic names for machine instructions) to directly control processor hardware. Each assembly instruction maps one-to-one to a machine code instruction. An **assembler** translates ALP source code to binary machine code.

1.1 Why ALP is Used in Embedded Systems

- **Maximum hardware control:** Direct access to all registers, flags, and ports.
- **Tight timing:** Exact instruction cycle count known — essential for time-critical routines (e.g., bit-bang communication, delay loops).
- **Minimum code size:** No compiler overhead; only absolutely necessary code is generated.
- **Boot code / startup:** Initial stack setup, clock configuration, and interrupt vector setup must be in assembly.
- **ISR optimization:** Critical ISRs are often hand-coded in assembly to minimize latency.

1.2 8051 ALP — Key Instructions

Instruction	Syntax / Example	Description
MOV	MOV A, #55H	Move immediate data to accumulator
ADD	ADD A, R0	Add R0 to Accumulator ($A = A + R0$)
SUBB	SUBB A, R1	Subtract R1 and Carry from A
SETB / CLR	SETB P1.0	Set/clear a specific bit
JMP / SJMP	SJMP LOOP	Unconditional short jump
JZ / JNZ	JZ DONE	Jump if accumulator is zero / not zero
DJNZ	DJNZ R2, LOOP	Decrement R2; jump if not zero (loop control)
LCALL / RET	LCALL DELAY	Long call subroutine / Return from subroutine
RETI	RETI	Return from interrupt (restores context)

8051 ALP Example: LED Toggle on Port 1

```

ORG 0000H          ; Program start address
MOV A, #0FFH       ; Load 0xFF into Accumulator (all bits high)
MAIN:
MOV P1, A          ; Send Accumulator to Port 1 (all LEDs ON)
ACALL DELAY        ; Call 1-second delay subroutine
CPL A              ; Complement Accumulator (0xFF -> 0x00)
SJMP MAIN          ; Repeat forever

DELAY:
MOV R0, #0FFH      ; Outer loop: 255 iterations
D1: MOV R1, #0FFH   ; Inner loop: 255 iterations
D2: DJNZ R1, D2      ; Decrement R1; loop until zero
    DJNZ R0, D1      ; Decrement R0; loop until zero
    RET              ; Return from subroutine
END

```

2. High-Level Language Programming — C for Embedded

C Language for Embedded Systems

C is the dominant high-level language for embedded systems programming. It provides a good balance between **hardware access** (via pointers, bitwise operations, volatile keyword) and **productivity** (structured code, functions, libraries). The C code is compiled by a cross-compiler for the target architecture.

2.1 Key C Concepts for Embedded Programming

Volatile Keyword

`volatile` tells the compiler that a variable's value can change at any time (e.g., by hardware or an ISR) and must not be cached in a register or optimized away:

```
volatile uint8_t flag = 0;    // Set by ISR, read by main loop
```

Bitwise Operations

Essential for setting, clearing, and testing individual bits in hardware registers:

Operation	C Code	Effect
Set a bit	<code>reg = (1 << n)</code>	Sets bit n without affecting others
Clear a bit	<code>reg &= ~(1 << n)</code>	Clears bit n without affecting others
Toggle a bit	<code>reg ^= (1 << n)</code>	Toggles bit n
Test a bit	<code>if (reg & (1 << n))</code>	Reads state of bit n

2.2 Processor Directives in Embedded C

Processor Directives

Processor directives (also called **preprocessor directives**) are instructions to the C preprocessor that run before actual compilation. They begin with `#` and control how the source code is processed.

Directive	Syntax	Purpose
#define	<code>#define LED_PIN 5</code>	Macro substitution; defines a constant or code snippet
#include	<code>#include "stm32f4xx.h"</code>	Includes a header file (device register definitions)
#ifdef / #endif	<code>#ifdef DEBUG ... #endif</code>	Conditional compilation (include code only for debug builds)
#pragma	<code>#pragma once</code>	Compiler-specific directive (prevent double-inclusion)
#ifndef	<code>#ifndef _HEADER_H</code>	Include guard to prevent multiple inclusions

2.3 Functions and Macros in Embedded C

- **Functions:** Reusable code blocks. Function calls add overhead (stack push/pop of parameters and return address). For embedded systems, frequently called short functions can be declared `inline` to avoid this overhead.
- **Macros:** Defined with `#define`. Expanded inline by the preprocessor — zero runtime overhead. Used for hardware register bit manipulation, delays, and port pin aliases.

C Macro vs. Inline Function Example

```
/* Macro -- no type safety, zero overhead */
#define SET_BIT(reg, bit) ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1U << (bit)))

/* Inline function -- type-safe, zero overhead (compiler expands inline) */
static inline void gpio_set(volatile uint32_t *reg, uint8_t bit) {
    *reg |= (1U << bit);
}

/* Usage */
SET_BIT(GPIOA->ODR, 5); /* Macro: sets PA5 */
gpio_set(&GPIOB->ODR, 3); /* Inline function: sets PB3 */
```

2.4 Embedded C++ Concept

Embedded C++ (EC++)

Embedded C++ is a subset of C++ designed for resource-constrained embedded targets. It uses C++ features that add **zero or minimal overhead** while providing better code organization and type safety compared to plain C.

C++ features commonly used in embedded:

- **Classes and Objects:** Encapsulate peripheral drivers (e.g., a UART class with `send()` and `receive()` methods).

- **Namespaces:** Prevent name collisions between libraries (e.g., `hal::uart::init()`).
- **Templates:** Generic, type-safe containers and algorithms (e.g., ring buffer template).
- **constexpr:** Compile-time constant evaluation reduces runtime overhead.
- **RAII (Resource Acquisition Is Initialization):** Automatic resource management (locks released when object goes out of scope — critical for RTOS mutexes).

C++ features AVOIDED in embedded (high overhead):

- Exceptions (`try/catch`) — adds large overhead and unpredictable timing.
- Runtime Type Information (RTTI) / `dynamic_cast` — requires type tables at runtime.
- `new/delete` (dynamic heap allocation) — causes fragmentation and non-deterministic timing.

3. Real-Time Operating System (RTOS)

RTOS — Real-Time Operating System

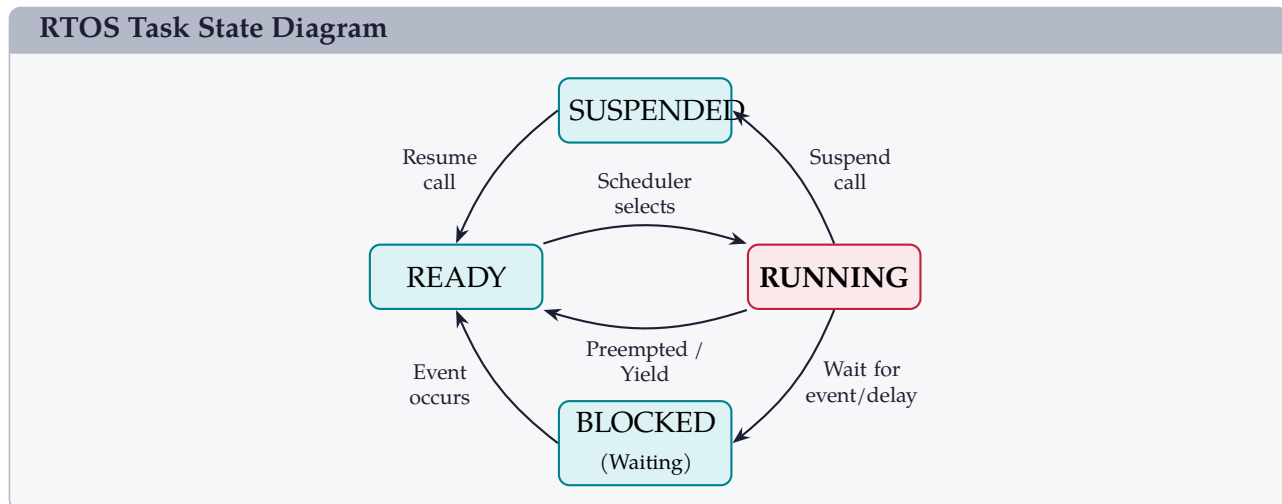
An **RTOS** is an operating system designed for **deterministic, time-bound execution** of tasks. It provides task scheduling, inter-task communication (queues, semaphores, mutexes), memory management, and timer services. Unlike general-purpose OS, an RTOS guarantees task completion within defined time limits.

3.1 OS Overview and Process Management

RTOS Services

- **Task (Thread) Management:** Create, delete, suspend, resume tasks with different priorities.
- **Scheduling:** Algorithm that decides which task runs on the CPU at any given time.
- **Inter-Task Communication (IPC):** Queues (FIFO data), semaphores (signaling), mutexes (shared resource protection).
- **Memory Management:** Static or dynamic allocation of memory blocks to tasks.
- **Timers:** Software timers for periodic callbacks without a dedicated hardware timer.
- **Examples:** FreeRTOS, Zephyr, RTEMS, VxWorks, uC/OS-II, ThreadX.

3.2 RTOS Task States



3.3 Task Scheduling Algorithms

1. Priority-Based Preemptive Scheduling

Priority-Based Preemptive Scheduling

Each task is assigned a **priority** (numeric value). The scheduler always runs the highest-priority task that is in the READY state. If a higher-priority task becomes ready while a lower-priority task runs, the lower-priority task is **immediately preempted** (stopped) and the higher-priority task takes the CPU.

- **Deterministic:** Highest-priority tasks always respond within bounded time.
- **Starvation risk:** Very low-priority tasks may never run if higher-priority tasks are always ready.
- **Priority Inversion:** A high-priority task is blocked because a low-priority task holds a shared resource. Solved by **Priority Inheritance Protocol**.
- **Used in:** FreeRTOS, VxWorks, RTEMS, most industrial RTOS.

2. Cyclic Scheduling (Time-Division Multiplexing)

Cyclic Scheduling

Tasks are arranged in a **fixed cyclic sequence** (also called a **static schedule**). A major cycle (hypercycle) is divided into **time slots**, and each task runs during its assigned slot. No dynamic priority decisions — the schedule is fixed at design time.

- **Predictable:** Exact execution times are known at compile time.
- **Simple:** No complex scheduler overhead.
- **Inflexible:** Cannot respond to dynamic events; hard to add new tasks.
- **Used in:** Safety-critical aerospace and automotive systems (ARINC 653, AUTOSAR).

3. Round-Robin Scheduling

Round-Robin Scheduling

All tasks of **equal priority** are given equal CPU time **time slices** (quantum, e.g., 10 ms). The scheduler cycles through all ready tasks in a circular queue. When a task's time slice expires, it is preempted and the next task in the queue runs.

- **Fair:** All equal-priority tasks share CPU time equally.
- **No starvation** for equal-priority tasks.
- **Response time:** Depends on the number of tasks and the time quantum (worst case: $(n - 1) \times Q$).
- **Used in:** Linux time-sharing, RTOS task groups at same priority.

3.4 Comparison of RTOS Scheduling Algorithms

Feature	Priority-Based	Cyclic	Round-Robin
Flexibility	High	Low	Medium
Determinism	High	Very High	Medium
Fairness	Low (priority skew)	Fixed	High (equal priority)
Overhead	Medium	Very Low	Low
Starvation	Possible	No	No (equal priority)
Best For	General RTOS	Safety-critical	Time-sharing tasks

3.5 Basic Design Rules Using RTOS

1. **Assign priorities carefully:** Critical, fast-response tasks (e.g., motor control) get the highest priority. Background tasks (e.g., logging) get the lowest.
2. **Use semaphores/mutexes** to protect shared resources (global variables, hardware peripherals) from concurrent access by multiple tasks.
3. **Avoid blocking calls in high-priority tasks:** Use non-blocking APIs or timeouts.
4. **Minimize stack usage:** Each task has its own stack. Over-allocating wastes RAM; under-allocating causes stack overflow (hard to debug).
5. **Keep ISRs minimal:** ISR sets a flag or sends to a queue; task processes the data.
6. **Use RTOS trace tools:** Tools like FreeRTOS+Trace or Segger SystemView to observe task switching, timing, and CPU usage.

Quick Revision — Module 4

Embedded Software Development & RTOS | PE-EC703A

Key Formulas at a Glance

8051 Delay Loop Cycles	$N_{cycles} = (\text{count}_{outer}) \times (\text{count}_{inner}) \times 5$ (approx., per DJNZ)
RTOS Response Time (RR)	$t_{response} \leq (n - 1) \times Q$ (n = tasks at same priority, Q = time quantum)
CPU Utilization Check	$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1$ (C_i = exec. time, T_i = period of task i)

One-Line Definitions

- **ALP:** Assembly Language Programming — low-level mnemonic-based programming for direct hardware control.
- **Assembler:** Translates assembly (mnemonic) code to binary machine code.
- **Cross-compiler:** Compiler on a host PC generating code for a different target (e.g., ARM GCC).
- **volatile:** C keyword that prevents compiler from caching a variable that can change externally (e.g., ISR flag).
- **Preprocessor Directive:** Instructions beginning with # that control compilation before the compiler runs.
- **Inline function:** C++ function expanded at call site by the compiler, eliminating function call overhead.
- **RTOS:** Operating system guaranteeing deterministic task scheduling with hard timing deadlines.
- **Task (Thread):** An independent unit of execution managed by the RTOS scheduler.
- **Preemption:** RTOS forcibly stopping a running task to run a higher-priority task.
- **Semaphore:** RTOS signaling mechanism for synchronization between tasks or between ISR and tasks.
- **Mutex:** Mutual exclusion object; protects shared resources from concurrent access (includes priority inheritance).
- **Priority Inversion:** A high-priority task blocked by a low-priority task holding a needed resource.
- **Round-Robin:** Scheduling algorithm giving equal time slices to all tasks at the same priority level.

Frequently Asked / Viva Questions**Q1. What is the difference between assembly language and C for embedded programming?**

ALP gives maximum hardware control, exact cycle-count timing, and minimum code size but is processor-specific and harder to maintain. C provides portability, structured code, and faster development with slightly larger code. Both are commonly mixed: C for application logic, ALP for startup and timing-critical routines.

Q2. What is the purpose of the `volatile` keyword in embedded C?

It tells the compiler that a variable can change unexpectedly (by an ISR, DMA, or hardware register) and must not be optimized or cached. Without `volatile`, the compiler may read the variable once and reuse the cached value, missing updates.

Q3. What are preprocessor directives? Give two examples.

Instructions beginning with `#` that are processed before compilation. Examples: `#define LED 5` (macro constant), `#include "stm32.h"` (file inclusion), `#ifdef DEBUG` (conditional compilation).

Q4. What is an RTOS and how does it differ from a bare-metal embedded system?

An RTOS provides multi-tasking, scheduling, and synchronization services. In bare-metal, a single `while(1)` loop and ISRs handle everything sequentially — no time guarantees between events. RTOS ensures tasks run within defined deadlines and can run concurrently.

Q5. Explain priority-based preemptive scheduling.

Each task has a priority number. The RTOS scheduler always runs the highest-priority READY task. If a higher-priority task becomes ready (e.g., unblocked by an ISR posting to its queue), the currently running lower-priority task is immediately preempted and the higher-priority task takes the CPU.

Q6. What is priority inversion and how is it solved?

Priority inversion: a high-priority task (H) is blocked waiting for a resource held by a low-priority task (L), while a medium-priority task (M) runs and prevents L from completing. **Solution:** Priority Inheritance — temporarily raise L's priority to H's level until it releases the resource.

Q7. Compare cyclic scheduling and round-robin scheduling.

Cyclic scheduling uses a fixed time-division slot sequence (fully deterministic, best for safety-critical systems, inflexible). Round-robin dynamically gives equal time quantum to all equal-priority tasks (fair, flexible, slight unpredictability in deadline, good for general tasks).

Q8. What is the difference between a semaphore and a mutex?

A **semaphore** is a signaling mechanism (binary or counting) primarily used for task synchronization and ISR-to-task signaling. A **mutex** (Mutual Exclusion) is specifically for protecting shared resources from concurrent access and includes priority inheritance to prevent priority inversion.

Frequently Asked / Viva Questions (contd.)

Q9. What is the role of the RTOS kernel?

The RTOS kernel manages task creation, deletion, scheduling (context switching), inter-task communication (queues, semaphores), timers, and memory allocation. It runs the context switch code that saves and restores task registers during task switching.

Q10. What are the advantages of C++ over C in embedded programming?

C++ provides encapsulation (classes for peripheral drivers), type safety (templates), compile-time computation (constexpr), and RAII for automatic resource management. These improve code organization and reliability with zero runtime cost if high-overhead features (exceptions, RTTI, heap) are avoided.

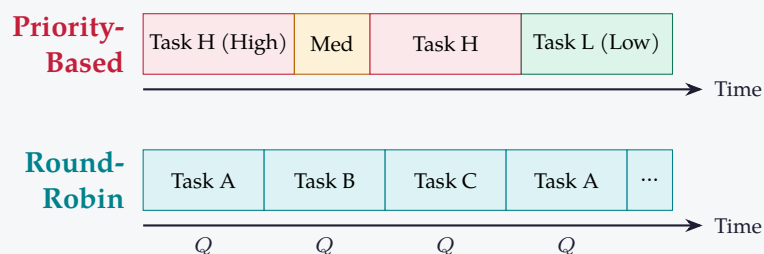
Q11. What is a time quantum in round-robin scheduling?

The time quantum (slice) is the maximum CPU time a task runs before being preempted and the next task getting a turn. Shorter quantum gives better responsiveness but more context-switch overhead. Typical values: 1–20 ms.

Q12. What is the DJNZ instruction in 8051 assembly and why is it used?

DJNZ Rn, label — Decrement Register *n* and Jump if Not Zero. It is the standard loop instruction: `MOV R0, #100; LOOP: DJNZ R0, LOOP` creates a 100-iteration delay loop with a 2-machine-cycle instruction — very efficient for timing loops.

RTOS Scheduling: Priority-Based vs. Round-Robin Timeline



Module 5

Introduction to Microchip PIC16 Family

1. Introduction to Microchip PIC16 Family

PIC Microcontroller

PIC (Peripheral Interface Controller) is a family of microcontrollers manufactured by **Microchip Technology**. PIC microcontrollers are known for their RISC-based Harvard architecture, a small instruction set (<35 instructions for mid-range PIC), low cost, wide voltage range (2.0–5.5 V), and abundant peripheral integration. They are widely used in hobbyist and industrial embedded applications.

1.1 PIC Architecture Classifications

Family	Program Word	Data Bus	Examples
PIC10 (Base)	12-bit	8-bit	PIC10F200, PIC10F220
PIC12 (Base)	12-bit	8-bit	PIC12F508, PIC12F675
PIC16 (Mid-range)	14-bit	8-bit	PIC16F84A, PIC16F877A, PIC16F873
PIC18 (Enhanced)	16-bit	8-bit	PIC18F4550, PIC18F452
PIC24 / dsPIC	24-bit	16-bit	dsPIC33, PIC24FJ
PIC32	32-bit MIPS	32-bit	PIC32MX, PIC32MZ

1.2 Key Architectural Features of PIC Mid-Range (PIC16)

- **Harvard Architecture:** Separate program (Flash) and data (RAM + SFR) buses.
- **RISC Core:** Only 35 instructions; most execute in 1 instruction cycle (4 clock cycles).
- **Instruction Cycle:** $T_{cy} = 4/f_{OSC}$ (e.g., at 20 MHz: $T_{cy} = 200$ ns).
- **8-bit Working Register (W):** Equivalent to the Accumulator in 8051.
- **File Register Architecture:** All RAM, SFRs, and I/O registers accessed via “file register” addresses.

- **Pipelining:** 2-stage pipeline (Fetch + Execute) — while one instruction executes, the next is fetched.

PIC16 Key Formulas

$$T_{cy} = \frac{4}{f_{OSC}} \quad (1 \text{ instruction cycle} = 4 \text{ clock periods})$$

$$\text{Timer0 Overflow Time} = \frac{(256 - \text{TMR0}) \times \text{Prescaler} \times 4}{f_{OSC}}$$

$$\text{ADC Resolution} = \frac{V_{REF}}{2^N - 1} \quad (N = 10 \text{ bits for PIC16F873})$$

$$\text{Baud Rate (UART)} = \frac{f_{OSC}}{64 \times (\text{SPBRG} + 1)} \quad (\text{Low speed mode})$$

2. PIC16F873 — Architecture and Features

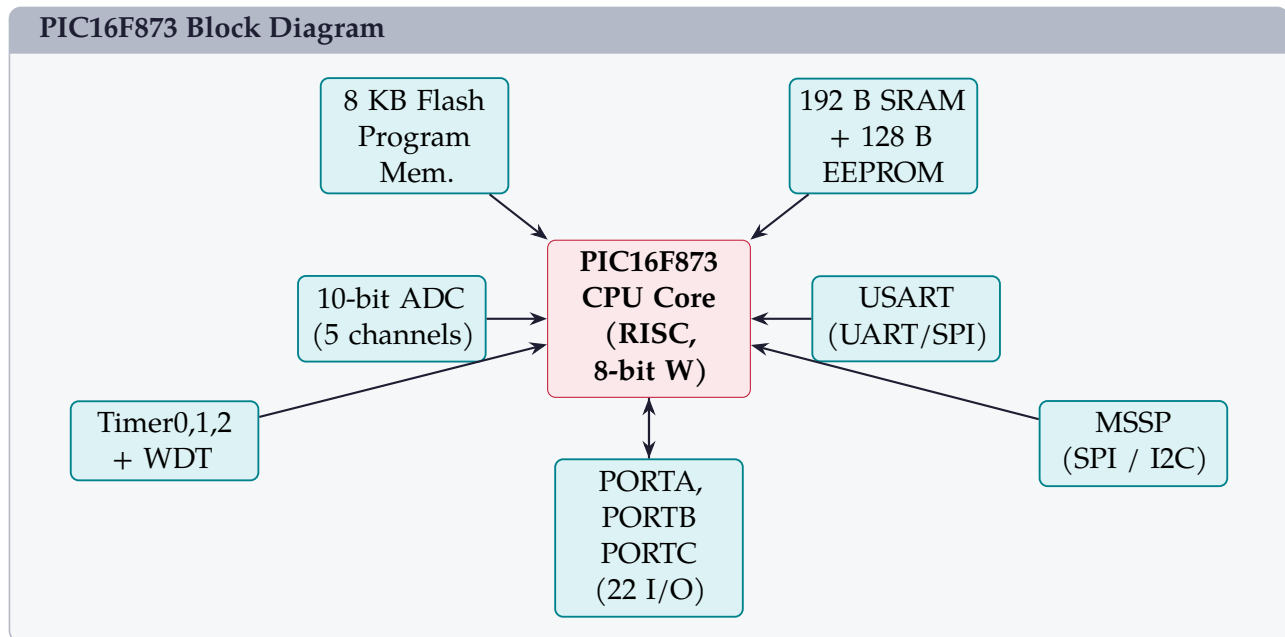
PIC16F873

The **PIC16F873** is a 28-pin Flash-based mid-range PIC microcontroller from Microchip. It is a popular choice for educational and industrial embedded designs due to its integrated analog and communication peripherals, Flash programmability, and low power consumption.

2.1 PIC16F873 — Key Specifications

Feature	Specification
Program Memory	8 KB Flash (4096 × 14-bit words)
Data Memory (RAM)	192 bytes SRAM
Data EEPROM	128 bytes
I/O Ports	PORTA (6 pins), PORTB (8 pins), PORTC (8 pins) = 22 I/O
Timers	Timer0 (8-bit), Timer1 (16-bit), Timer2 (8-bit + PR2)
ADC	5-channel, 10-bit ADC (AN0–AN4 on PORTA)
Serial Ports	USART (UART/SPI async), MSSP (SPI + I2C)
Interrupts	Up to 13 interrupt sources
Clock Speed	DC to 20 MHz (using external crystal)
Supply Voltage	2.0 V to 5.5 V
Package	28-pin PDIP, SOIC, SSOP
Watchdog Timer	Built-in WDT with independent RC oscillator

PIC16F873 Block Diagram



3. On-Chip Peripherals of PIC16F873

3.1 Watchdog Timer (WDT)

- Has its own internal **RC oscillator** (independent of main clock); continues running even if main clock fails.
- Default timeout period ≈ 18 ms; extended via **postscaler** (up to $\times 128 \approx 2.3$ s).
- Reset by `CLRWDT` instruction in software.
- On timeout: generates a device reset (MCLR pin goes low momentarily).
- Can be enabled/disabled via configuration word bits.

3.2 ADC (Analog-to-Digital Converter)

- **10-bit successive approximation ADC**: Converts analog voltage to a 10-bit digital value (0 to 1023).
- **5 analog input channels** (AN0–AN4) on PORTA (multiplexed with digital I/O).
- **Reference voltage**: V_{REF+} and V_{REF-} pins (or V_{DD} and V_{SS} by default).
- **Acquisition time**: Capacitor sampling time before conversion starts; minimum $2 \mu\text{s}$.
- **Result format**: Left-justified or right-justified; stored in ADRESH:ADRESL registers.

PIC16F873 ADC — Conversion Steps

1. Configure ADCON1 (select analog/digital pin assignment and V_{REF}).
2. Configure ADCON0 (select channel AN x , clock source, turn ADC ON).
3. Wait for acquisition time ($\geq 2 \mu\text{s}$).
4. Set GO/DONE bit in ADCON0 to start conversion.

5. Poll `GO/DONE` bit until it clears (conversion complete, $\approx 12 \times T_{AD}$).
6. Read 10-bit result from `ADRESH` (high 2 bits) and `ADRESL` (low 8 bits).

3.3 Asynchronous Serial Port — USART

- Full-duplex UART on RC6 (TX) and RC7 (RX).
- Baud rate controlled by SPBRG register and BRGH bit (high/low speed).
- **High-speed mode (BRGH=1):** $\text{Baud} = f_{OSC} / (16 \times (\text{SPBRG} + 1))$
- **Low-speed mode (BRGH=0):** $\text{Baud} = f_{OSC} / (64 \times (\text{SPBRG} + 1))$
- Interrupt flags: TXIF (transmit buffer empty), RCIF (receive buffer full).

3.4 SPI and I2C — MSSP Module

- The **MSSP** (Master Synchronous Serial Port) supports both SPI and I2C modes.
- **SPI Mode:** Uses RC3 (SCK), RC4 (SDI), RC5 (SDO), and any GPIO as CS (Chip Select). Master or slave configurable.
- **I2C Mode:** Uses RC3 (SCL) and RC4 (SDA) with open-drain outputs. Supports 7-bit and 10-bit addressing.

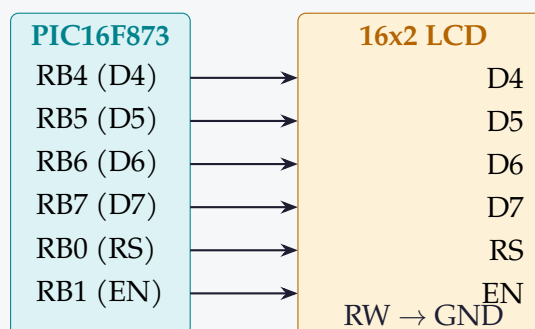
4. Interfacing with PIC16F873

4.1 LCD Interfacing

LCD (16×2) Interfacing with PIC16F873

A standard Hitachi HD44780-compatible 16×2 character LCD is interfaced using **4-bit mode** (4 data lines + 3 control lines) to save I/O pins. Control pins: RS (Register Select: 0=Command, 1=Data), RW (Read/Write, tied to GND for write-only), EN (Enable pulse to latch data).

PIC16F873 to 16×2 LCD — 4-bit Interface



4.2 ADC Interfacing (Sensors)

- Temperature sensor (LM35): Output voltage = 10 mV/°C. Connect to AN0.

– ADC result $\rightarrow$ Temperature: $T(^{\circ}C) = \frac{ADC \times V_{REF}}{1023 \times 0.01}$

- Potentiometer / Joystick: Directly connected to analog input (voltage divider).
- Light sensor (LDR): Used in voltage divider; ADC reads light level.

4.3 Stepper Motor Interfacing

- A stepper motor advances by a fixed angle (step) for each control pulse.
- **Step angle:** Depends on motor type (e.g., $1.8^{\circ}/\text{step} = 200 \text{ steps/revolution}$).
- **Driver IC:** L298N or ULN2003 connects between PIC output pins and motor coils.
- PIC sends sequence of bit patterns to PORTB to rotate the stepper.
- **Full-step sequence (4-phase):** 1100 $\rightarrow$ 0110 $\rightarrow$ 0011 $\rightarrow$ 1001 (repeat).

4.4 Keyboard (Keypad) Interfacing

- A **4 $\times$ 4 matrix keypad** uses 4 row + 4 column lines = 8 pins.
- PIC drives rows LOW one at a time, then reads which column line goes LOW.
- The row/column combination identifies the pressed key.
- Debouncing: Software delay ($\sim 20 \text{ ms}$) after detecting key press before confirming it.

4.5 DAC (Digital-to-Analog Converter)

- PIC16F873 has no built-in DAC. External options:
 - **R-2R Resistor Ladder:** Passive DAC using resistors connected to PORTB (8-bit). Simple, low cost, no extra IC.
 - **External DAC IC (MCP4921):** 12-bit DAC with SPI interface.
 - **PWM + RC filter:** Use Timer2 PWM output, filter with RC low-pass $\rightarrow$ analog voltage proportional to duty cycle.

5. Case Studies of Embedded System Design

5.1 Case Study 1: Automated Chocolate Vending Machine

Automated Chocolate Vending Machine using PIC16F873

Problem: Design a vending machine that accepts coins (detected by IR sensor), displays balance on LCD, dispenses chocolate (stepper motor) when enough coins are inserted, and provides change.

Hardware Components:

- PIC16F873 (controller), 16 $\times$ 2 LCD (display), IR coin sensor on AN0 (ADC), Stepper motor + ULN2003 (dispenser), Buzzer on RB0 (feedback), Keypad for item selection.

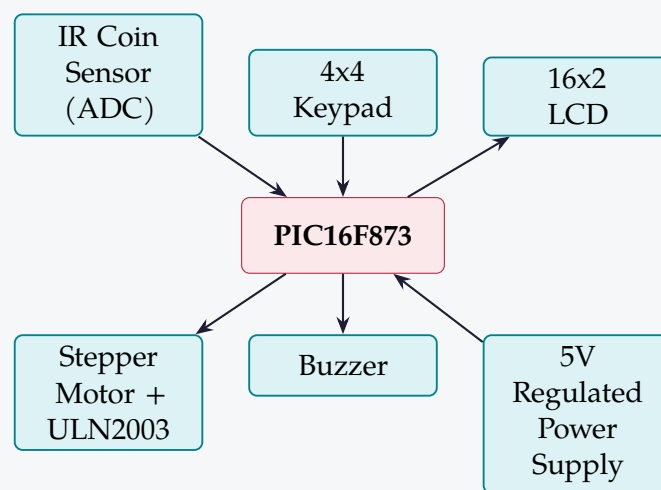
Software Flow:

1. Initialize LCD, ADC, I/O ports.

2. Display “Insert Coin” on LCD.
3. Loop: Poll coin sensor (ADC reading > threshold → coin detected).
4. On coin: Increment balance; update LCD display.
5. If balance $\geq$ price: Drive stepper motor (4 steps) to dispense; beep buzzer.
6. Calculate and dispense change (reverse stepper? No — separate change mechanism).
7. Reset balance; display “Thank You”.

Key Programming Concepts Used: ADC polling, stepper motor control sequences, LCD 4-bit mode, software debounce, timer-based delay.

Vending Machine System Block Diagram



5.2 Case Study 2: Digital Camera Controller

Digital Camera Controller using PIC (Concept)

Note: A full digital camera uses high-performance processors (e.g., ARM Cortex-A) for image processing. PIC16F873 can implement the **camera control subsystem** (not image processing).

PIC16F873 Role in Digital Camera:

- **Shutter Control:** Timer-based precise shutter speed control (1/1000s to 1/30s) using Timer1 interrupt.
- **Focus Motor Control:** Stepper motor drives lens position; PIC reads ADC from autofocus sensor.
- **Flash Trigger:** GPIO pin triggers external flash unit via optocoupler.
- **User Interface:** Reads mode dial (keypad), displays settings on small LCD.
- **Battery Monitor:** ADC reads battery voltage divider; displays charge level.
- **Serial Communication:** UART or SPI to main image processor (ARM) for command passing.

Key Techniques: Timer interrupts for precision timing, ADC for sensor reading, SPI for inter-processor communication, WDT for reliability.

Quick Revision — Module 5

Embedded System Design using PIC Microcontroller | PE-EC703A

Key Formulas at a Glance

Instruction Cycle	$T_{cy} = \frac{4}{f_{OSC}}$ (at 20 MHz: $T_{cy} = 200$ ns)
Timer0 Overflow	$t_{TMR0} = \frac{(256 - TMR0_init) \times \text{Prescaler} \times 4}{f_{OSC}}$
ADC Result (10-bit)	$V_{in} = \frac{ADC \times V_{REF}}{1023}$ (right-justified)
UART Baud (BRGH=1)	$\text{Baud} = \frac{f_{OSC}}{16 \times (\text{SPBRG} + 1)}$
ADC Temperature (LM35)	$T(^{\circ}\text{C}) = \frac{ADC \times V_{REF}}{1023 \times 0.01}$

One-Line Definitions

- **PIC:** Peripheral Interface Controller — Microchip Technology's RISC-based MCU family.
- **PIC16F873:** 28-pin, 8-bit Flash PIC MCU; 8 KB Flash, 192 B SRAM, 10-bit ADC, USART, MSSP.
- **W Register:** Working Register in PIC — equivalent to Accumulator; all ALU operations use W.
- **File Register:** PIC's general-purpose RAM + SFR memory, accessed via file register addresses.
- **ADCON0/1:** ADC Control registers in PIC16F873 that configure channel selection, clock, and reference.
- **MSSP:** Master Synchronous Serial Port — supports SPI and I2C in PIC16F873.
- **USART:** Universal Synchronous/Asynchronous Receiver/Transmitter — UART + SPI on PIC.
- **SPBRG:** Serial Port Baud Rate Generator register — sets UART baud rate in PIC.
- **Prescaler:** Divides the clock signal before reaching the timer — extends the timer overflow period.
- **TMR0:** 8-bit Timer/Counter in PIC16F873 (overflows at 255 → 0, generates TMR0IF).
- **CLRWDT:** Assembly instruction to reset the Watchdog Timer in PIC.
- **Full-step sequence:** 4-phase stepper motor control pattern (1100, 0110, 0011, 1001).

Frequently Asked / Viva Questions

Q1. What does PIC stand for and who manufactures it?

PIC stands for **Peripheral Interface Controller**. It is manufactured by **Microchip Technology Inc.**

Q2. State the key features of PIC16F873.

28-pin PDIP; 8 KB Flash program memory; 192 B SRAM; 128 B EEPROM; 10-bit 5-channel ADC; three timers (Timer0/1/2); USART; MSSP (SPI + I2C); up to 20 MHz clock; 2.0–5.5 V supply; built-in WDT.

Q3. What is the W register in PIC? How is it different from 8051's Accumulator?

The **W (Working) register** is PIC's primary data register used for ALU operations. Similar to the Accumulator in 8051, it is used as the source/destination for most arithmetic and logic instructions. Unlike 8051's A, W is not directly addressable in the file register map.

Q4. What is the instruction cycle of PIC16 and how does it relate to clock frequency?

One instruction cycle = 4 clock cycles: $T_{cy} = 4/f_{OSC}$. At 20 MHz, $T_{cy} = 200$ ns. Most PIC instructions execute in 1 cycle (200 ns); branch instructions take 2 cycles.

Q5. How does the PIC16F873 ADC work? What is the resolution?

It is a 10-bit successive approximation ADC. It samples the analog voltage on AN0–AN4, converts it to a 10-bit number (0–1023) representing V_{in} between V_{REF-} and V_{REF+} . Resolution = $V_{REF}/1023$ (e.g., ≈ 4.88 mV for 5V reference).

Q6. Explain how a stepper motor is interfaced with PIC16F873.

The PIC outputs a 4-bit step sequence to PORTB (e.g., 1100, 0110, 0011, 1001 for full-step mode). Each pattern energizes the motor coils in sequence. A ULN2003 Darlington driver IC between PORTB and the motor coils amplifies the current. The step rate (delay between patterns) controls motor speed.

Q7. What is the difference between USART and MSSP in PIC16F873?

USART: Handles asynchronous serial (UART) and synchronous serial (SPI) communication on RC6/RC7. **MSSP:** A second serial module supporting both SPI (master/slave) and I2C master/slave modes on RC3/RC4/RC5. Both can be used simultaneously.

Q8. How is LCD interfaced with PIC16F873 in 4-bit mode?

Only 4 data lines (D4–D7) are connected to PORTB, along with RS and EN control lines (RW tied to GND). Data is sent as two 4-bit nibbles (high nibble first, then low nibble), each latched by pulsing EN high then low.

Q9. How does the Watchdog Timer work in PIC16F873?

The WDT has its own RC oscillator (≈ 18 ms period). Software must execute `CLRWDT` instruction before timeout. If not cleared in time (e.g., due to program hang), WDT causes a device reset. Timeout can be extended using a postscaler (up to $\times 128 \approx 2.3$ s).

Frequently Asked / Viva Questions (contd.)

Q10. What are the Timer modes available in PIC16F873?

Timer0: 8-bit timer/counter with prescaler (1:2 to 1:256). **Timer1:** 16-bit timer/counter, can use external crystal for low-power RTC. **Timer2:** 8-bit timer with prescaler and postscaler; period register PR2 sets overflow point; used for PWM generation.

Q11. Describe the design of an automated vending machine using PIC16F873.

- IR sensor (ADC) detects coins; balance tracked in SRAM variable.
- LCD (4-bit mode) shows balance and prompts.
- Keypad selects product; price compared with balance.
- If balance $\geq$ price: stepper motor (via ULN2003) dispenses product; buzzer beeps.
- Balance decremented by price; “Thank You” displayed.

Q12. Why is PIC preferred for cost-sensitive embedded applications?

PIC MCUs are very inexpensive (PIC16F84A ~ Rs. 80–160), available in small packages, have wide voltage range (2–5.5 V), abundant development tools (MPLAB IDE, free compiler), large community, and integrate all needed peripherals on-chip (eliminating external ICs). They are ideal for high-volume, cost-sensitive consumer products.

PIC16F873 vs. 8051 vs. ARM Cortex-M0 — Comparison

Feature	PIC16F873	8051 (AT89S52)	ARM Cortex-M0
Architecture	Harvard RISC (8-bit)	Harvard CISC (8-bit)	Harvard RISC (32-bit)
Instructions	35 (14-bit words)	255 (variable)	Thumb-2 (16/32-bit)
Max Clock	20 MHz	33 MHz	48 MHz
Flash	8 KB	8 KB (ISP)	32–128 KB
RAM	192 bytes	256 bytes	8–32 KB
Built-in ADC	10-bit, 5-ch	None	12-bit, 8+ ch
Price (approx.)	Rs. 40–160	Rs. 65–160	Rs. 80–400
IDE	MPLAB X	Keil, SDCC	Keil, GCC ARM